

编译原理·语法分析导论、Parser 与 CFG

Syntax Analysis: Intro & Parser & CFG ——详细中文学习笔记

基于课件 3.Syntax Analysis_Intro_Parser_CFG.pdf (中山大学·赵帅) 整理

本笔记说明

本笔记覆盖课件第 3 讲共 55 页内容，主线是：

token 流 $\xrightarrow{\text{语法分析 Parser}}$ 语法结构 Parse Tree / AST

先说明为什么正则语言不足以描述程序语法，再系统学习文法(Grammar)、上下文无关文法(Context-Free Grammar, CFG)、推导(Derivation)、分析树(Parse Tree)、二义性(Ambiguity)和乔姆斯基文法体系(Chomsky Grammar Hierarchy)。每个重要知识点都标注课件页码 [p 页码]，并补充通俗解释与复习用例。

目录

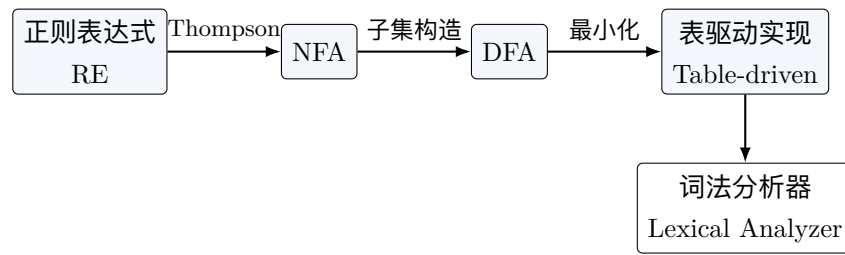
1 从词法分析走向语法分析 (第 2–5 页)	2
1.1 前置回顾：词法分析的实现链路 [p2,5]	2
1.2 正则语言的局限 [p3,4]	3
1.3 为什么需要语法分析 [p4,5]	3
2 语法分析在编译器中的位置 (第 6–11 页)	4
2.1 编译阶段中的 Parser [p6]	4
2.2 语法分析的定义 [p9,10]	4
2.3 语法分析举例 [p11]	4
3 如何定义语法：形式语言与文法 (第 12–16 页)	5
3.1 自然语言与形式语言 [p12,13]	5
3.2 文法的四元组定义 [p14]	5
3.3 产生式与 BNF [p15]	6
3.4 中文句子的文法例子 [p16]	6
4 上下文无关文法 CFG (第 17–23 页)	6
4.1 为什么 CFG 适合程序语法 [p17]	6
4.2 CFG 的形式化定义 [p18]	7
4.3 算术表达式 CFG [p18,19,23]	7
4.4 文法书写约定 [p20–23]	7

1 从词法分析走向语法分析 (第 2–5 页)	2
5 推导：从文法到语言 (第 24–30 页)	8
5.1 推导的基本定义 [p25]	8
5.2 句型、句子与语言 [p26]	8
5.3 例子：A $\rightarrow$ c Ab 生成的语言	9
5.4 推导过程中的选择 [p28,29]	9
5.5 最左推导与最右推导 [p29,30]	9
6 分析树 Parse Tree (第 31–35 页)	10
6.1 分析树的定义 [p32,33]	10
6.2 叶子、产出与边缘 [p34,35]	10
6.3 小型分析树示意	11
7 二义性与消歧 (第 36–41 页)	11
7.1 二义性的基本问题 [p36,37]	11
7.2 文法二义性与语言二义性 [p38]	11
7.3 二义性判定的困难 [p39]	12
7.4 用优先级和结合性消除表达式二义性 [p40,41]	12
8 乔姆斯基文法体系 (第 42–49 页)	12
8.1 四类文法总览 [p42,43]	12
8.2 Type 0: 无限制文法 [p44]	13
8.3 Type 1: 上下文有关文法 [p45]	13
8.4 Type 2: 上下文无关文法 [p46]	14
8.5 Type 3: 正则文法 [p47,48]	14
8.6 四类文法对比 [p49]	15
9 实践中的语法分析与 AST (第 50–55 页)	15
9.1 为什么编译器仍大量使用 CFG [p50]	15
9.2 Parsing 到底做什么 [p51]	15
9.3 Parse Tree vs AST [p52]	16
9.4 课程总结 [p53–55]	16
10 练习题与参考答案	17

1 从词法分析走向语法分析 (第 2–5 页)

1.1 前置回顾：词法分析的实现链路 [p2,5]

上一讲已经完成了从词法规范到词法分析器实现的主线：



- **Thompson 算法** (Thompson Algorithm)：把正则表达式转换为 NFA。
- **子集构造法** (Subset-Construction Algorithm)：把 NFA 转换为 DFA。
- **分割法** (Partition Algorithm)：最小化 DFA，减少状态数。
- 词法分析的输出是 token 流；本讲从这里继续，研究 token 流是否组成合法程序结构。

1.2 正则语言的局限 [p3,4]

正则语言 (Regular Language) 的能力边界

正则语言适合描述局部、线性、无需记忆的模式，例如标识符、数字、字符串、注释等 token。但它不能表达需要任意深度记忆或配对计数的结构。

课件强调了几个典型例子：

- $L = \{a^n b^n \mid n \geq 1\}$ 不是正则语言：识别它需要记住前面读了多少个 a ，再检查后面是否有同样数量的 b 。有限自动机状态数有限，不能无限计数。
- $L = \{a^n b a^n \mid n \geq 0\}$ 也体现同类困难：中间的 b 左右两侧 a 的数量要相等。
- 括号匹配、花括号匹配 $\{\{\{\cdots\}\}\}$ 不是正则语言，因为括号可以任意嵌套，必须记住尚未匹配的左括号数量。
- 嵌套结构，例如 `if ... if ... else ... else`、C 语言中的嵌套 `for` 循环，通常不能只靠正则语言描述。

直观理解

有限自动机像一个只有有限记忆的小机器。它可以判断“当前是不是正在读标识符”，但无法记住“已经读过 137 个左括号，所以后面必须出现 137 个右括号”。因此，词法分析可以用正则语言；语法分析需要更强的形式化工具。

1.3 为什么需要语法分析 [p4,5]

程序设计语言的 token 本身通常可以由正则表达式描述；但是 token 之间如何组合成表达式、语句、函数、块结构，则需要能够描述递归结构和嵌套结构的模型。

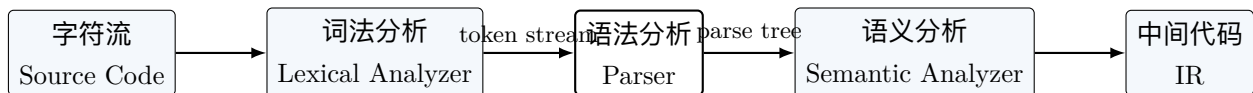
Regular Language for Lexical Analysis Context-Free Language for Syntax Analysis

这就是本讲引入上下文无关文法 (Context-Free Grammar, CFG) 的核心原因。

2 语法分析在编译器中的位置 (第 6–11 页)

2.1 编译阶段中的 Parser [p6]

语法分析是编译器前端的第二个主要阶段，位于词法分析之后、语义分析之前。



词法分析器把源代码拆成 token 流，例如 [p7]：

```
while (y<z){ int x=a+b; y+=x; }
```

会被词法分析为类似

```
(keyword,while), (id,y), (sym,<), (id,z), ...
```

这样的 token 序列。语法分析器接手后，不再关心字符级细节，而是检查这些 token 是否构成合法语句。

2.2 语法分析的定义 [p9,10]

语法分析 (Syntax Analysis / Parsing)

语法分析是编译的第二阶段，也称为**解析** (Parsing) 或**语法分析器** (Parser)。它从词法分析器获得 token 名组成的串，并验证该 token 串是否能由源语言的文法生成。如果合法，它构造**分析树** (Parse Tree) 并传给编译器前端后续阶段；如果非法，它报告语法错误。

更形式化地说，Parser 要完成两件事：

1. **判定成员关系**：输入 token 串 w 是否属于某个文法 G 生成的语言 $L(G)$ 。
2. **构造结构表示**：如果 $w \in L(G)$ ，构造它的推导结构，通常是 parse tree 或 AST。

2.3 语法分析举例 [p11]

例 1：源程序输入：

```
if (x == y) stmt1 else stmt2
```

词法分析输出类似：

```
KEY(IF) '(' ID(x) OP('==') ID(y) ')' ... KEY(ELSE) ...
```

Parser 要判断它是否符合 if 语句的语法，并生成表示条件、then 分支、else 分支的树形结构。

例 2: token 串

`<id,x> <op,*> <op,%>`

在 C 语言中不是合法语句。虽然每个 token 单独都可能合法，但它们的组合 `x *%` 不符合表达式语法。

关键区分

词法分析回答“每一段字符是什么 token”；语法分析回答“这些 token 能不能按语言规则组成句子”。合法 token 的任意拼接不一定是合法程序。

3 如何定义语法：形式语言与文法（第 12–16 页）

3.1 自然语言与形式语言 [p12,13]

- **自然语言** (Natural Language)：人类日常使用的语言，例如中文、英语。自然语言常有歧义，规则也可能依赖语境。
- **形式语言** (Formal Language)：由精确数学规则或机器可处理公式定义的语言，具有严格语法规则。
- **程序设计语言** (Programming Language)：一种形式语言，用于定义计算机程序。

形式语言可以用多种方式定义，例如**正则表达式** (Regular Expression)、**有限自动机** (Finite Automata) 和**文法** (Grammar)。其中 RE/FA 不够强，无法完整描述程序语法；文法尤其适合描述嵌套结构。

3.2 文法的四元组定义 [p14]

文法 (Grammar)

文法是定义语言的数学模型。一个文法通常写作

$$G = (V_T, V_N, S, \delta)$$

其中 V_T 是终结符集合， V_N 是非终结符集合， S 是开始符号， δ 是产生式集合，并且 $V_T \cap V_N = \emptyset$ 。

组成	英文术语	含义
V_T	Terminals	终结符 (terminal symbols)，构成最终字符串的基本符号。对编译器而言，终结符本质上通常就是 token，位于 parse tree 的叶子。

V_N	Non-terminals	非终结符 (non-terminal symbols), 表示一类可继续展开的语法结构, 如 expression、statement、loop, 通常位于 parse tree 的内部节点。
S	Start Symbol	开始符号 (start symbol), 推导的根, 是某个非终结符。课件要求 S 至少出现在某条产生式左部一次。
δ	Productions	产生式 (productions) 集合, 规定终结符和非终结符如何组合成句子。

3.3 产生式与 BNF [p15]

产生式 (Production)

产生式形如

$$\text{LHS} \rightarrow \text{RHS}$$

读作“左部产生右部”或“左部定义为右部”。左部称为**产生式头** (head / left-hand side), 右部称为**产生式体** (body / right-hand side)。→ 有时也写作::=, 这就是常见的**巴科斯范式** (Backus–Naur Form, BNF) 记法。

产生式的直观含义是：当推导过程中遇到左部符号时, 可以把它替换为右部符号串。

3.4 中文句子的文法例子 [p16]

课件给出一个简化中文句子文法：

$$\begin{aligned} \langle \text{句子} \rangle &\rightarrow \langle \text{主语} \rangle \langle \text{谓语} \rangle \langle \text{宾语} \rangle \\ \langle \text{主语} \rangle &\rightarrow \text{我} \mid \text{猫} \\ \langle \text{谓语} \rangle &\rightarrow \text{喜欢} \mid \text{追} \\ \langle \text{宾语} \rangle &\rightarrow \text{巧克力} \mid \text{老鼠} \end{aligned}$$

其中：

$$V_N = \{ \langle \text{句子} \rangle, \langle \text{主语} \rangle, \langle \text{谓语} \rangle, \langle \text{宾语} \rangle \}$$

$$V_T = \{ \text{我}, \text{猫}, \text{喜欢}, \text{追}, \text{巧克力}, \text{老鼠} \}, \quad S = \langle \text{句子} \rangle$$

它可以生成“猫喜欢巧克力”“我追老鼠”等句子。注意“老鼠追猫”若要被生成, 则“老鼠”还必须能作为主语、“猫”还必须能作为宾语；这说明文法必须**精确且完整**, 否则会漏掉本应合法的句子或生成不应合法的句子。

4 上下文无关文法 CFG (第 17–23 页)

4.1 为什么 CFG 适合程序语法 [p17]

检查程序是否**良构** (well-formed), 需要对合法程序结构给出规范。这个规范应满足：

- **精确**：不能靠自然语言猜测。
- **完备**：覆盖语言所有语法细节。
- **便于使用**：语言设计者和编译器实现者都能使用。

上下文无关文法 (Context-Free Grammar, CFG) 正好满足这些需求。它足够描述当代大多数编程语言的主要语法结构，尤其是表达式、语句、块、嵌套结构等递归语法。

4.2 CFG 的形式化定义 [p18]

上下文无关文法 (Context-Free Grammar, CFG)

CFG 仍写作四元组 $G = (V_T, V_N, S, \delta)$ ，但产生式必须形如

$$A \rightarrow \alpha$$

其中 $A \in V_N$ 是**单个非终结符**， $\alpha \in (V_N \cup V_T)^*$ 是终结符和非终结符组成的串。

“上下文无关”的意思是：只要看到非终结符 A ，就可以把它替换成 α ，不需要查看 A 左右是什么上下文。

4.3 算术表达式 CFG [p18,19,23]

课件先给出一个只含 +、*、括号和标识符 i 的表达式文法：

$$G = (\{i, +, *, (,)\}, \{E\}, E, \delta)$$

$$E \rightarrow i \mid E + E \mid E * E \mid (E)$$

这是一个能描述基本表达式的 CFG，但它没有规定 + 和 * 的优先级，也没有规定结合性，因此后面会产生二义性。

课件后面给出更常用的表达式文法：

$$E \rightarrow E + T \mid E - T \mid T$$

$$T \rightarrow T * F \mid T / F \mid F$$

$$F \rightarrow (E) \mid id$$

这里的 E, T, F 分别表示 expression、term、factor。它天然表达了“乘除优先于加减”：因为 + 和 - 只在 E 层出现，* 和 / 在更低的 T 层出现。

4.4 文法书写约定 [p20–23]

符号	常见含义
a, b, c 等靠前小写字母	终结符。
$+, *, (,)$, 标点、数字	终结符。

id, if 等粗体字符串	一个整体终结符，通常对应一个 token。
<i>A, B, C</i> 等靠前大写字母	非终结符。
<i>S</i>	通常表示开始符号。
<i>expr, stmt</i> 等小写斜体名字	非终结符，表示程序构造。
<i>E, T, F</i>	常用于表达式文法：表达式、项、因子。
<i>X, Y, Z</i> 等靠后大写字母	文法符号，可以是终结符或非终结符。
<i>u, v, ..., z</i> 等靠后小写字母	可能为空的终结字符串。
α, β, γ 等希腊字母	可能为空的文法符号串。

共享同一左部的规则可以合并：

$$\alpha \rightarrow \beta_1, \quad \alpha \rightarrow \beta_2, \quad \dots, \quad \alpha \rightarrow \beta_n$$

可写作：

$$\alpha \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

其中每个 β_i 称为一个**候选式** (alternative)。

默认情况下，如果未特别说明，第一条产生式的左部就是开始符号。

5 推导：从文法到语言（第 24–30 页）

5.1 推导的基本定义 [p25]

推导 (Derivation)

推导是连续应用产生式规则的过程。若存在产生式 $A \rightarrow \gamma$ ，并且某个文法符号串中出现了 A ，则

$$\alpha A \beta \Rightarrow \alpha \gamma \beta$$

表示“一步推导”，即把中间的 A 替换成 γ 。

常用记号：

- $\alpha \Rightarrow \beta$ ：一步推导。
- $\alpha \Rightarrow^* \beta$ ：零步或多步推导；因此任意 $\alpha \Rightarrow^* \alpha$ 成立。
- $\alpha \Rightarrow^+ \beta$ ：一步或多步推导。
- 若 $\alpha \Rightarrow^* \beta$ 且 $\beta \Rightarrow^* \gamma$ ，则 $\alpha \Rightarrow^* \gamma$ 。

5.2 句型、句子与语言 [p26]

句型、句子、语言

设 S 是文法 G 的开始符号。

- 若 $S \Rightarrow^* \alpha$ ，则 α 是 G 的**句型** (sentential form)。句型可以包含终结符和非终结符，也可以为空。
- 若某个句型不含任何非终结符，则称为**句子** (sentence)。
- 文法生成的所有句子的集合称为该文法生成的**语言** (language)：

$$L(G) = \{w \mid S \Rightarrow^* w, w \in V_T^*\}$$

5.3 例子： $A \rightarrow c \mid Ab$ 生成的语言 [p27]

文法：

$$G[A]: \quad A \rightarrow c \mid Ab$$

可以推导：

$$A \Rightarrow c$$

$$A \Rightarrow Ab \Rightarrow cb$$

$$A \Rightarrow Ab \Rightarrow Abb \Rightarrow cbb$$

$$A \Rightarrow Ab \Rightarrow Abb \Rightarrow Abbb \Rightarrow cbbb$$

因此它生成的语言是：

$$L(G) = \{cb^n \mid n \geq 0\}$$

也就是一个 c 后面跟任意多个 b 。

5.4 推导过程中的选择 [p28,29]

每一步推导通常有两个选择：

1. 替换哪个非终结符。
2. 对这个非终结符使用哪条产生式。

这两个选择直接对应 parser 的两大思路：

- 自顶向下分析 (Top-down parsing)：从开始符号出发，尝试生成输入串。
- 自底向上分析 (Bottom-up parsing)：从输入串出发，尝试规约回开始符号。

5.5 最左推导与最右推导 [p29,30]

最左推导与最右推导

最左推导 (Leftmost Derivation)：每一步总是替换当前句型中最左边的非终结符。

最右推导 (Rightmost Derivation)：每一步总是替换当前句型中最右边的非终结符。

以课件文法

$$E \rightarrow T \mid E + T, \quad T \rightarrow F \mid T * F, \quad F \rightarrow (E) \mid i$$

推导($i * i + i$) 为例。

最左推导：

$$\begin{aligned} E &\Rightarrow T \Rightarrow F \Rightarrow (E) \Rightarrow (E + T) \Rightarrow (T + T) \\ &\Rightarrow (T * F + T) \Rightarrow (F * F + T) \Rightarrow (i * F + T) \\ &\Rightarrow (i * i + T) \Rightarrow (i * i + F) \Rightarrow (i * i + i) \end{aligned}$$

最右推导：

$$\begin{aligned} E &\Rightarrow T \Rightarrow F \Rightarrow (E) \Rightarrow (E + T) \Rightarrow (E + F) \\ &\Rightarrow (E + i) \Rightarrow (T + i) \Rightarrow (T * F + i) \\ &\Rightarrow (T * i + i) \Rightarrow (F * i + i) \Rightarrow (i * i + i) \end{aligned}$$

两种推导替换顺序不同，但最终可对应同一棵分析树。

6 分析树 Parse Tree (第 31–35 页)

6.1 分析树的定义 [p32,33]

分析树 (Parse Tree)

分析树是推导的图形化表示。它保留“用了哪些产生式、形成了什么层次结构”，但过滤掉“推导过程中先替换哪个非终结符”的时间顺序。

分析树的结构规则：

- 每个内部节点表示一次产生式应用。
- 内部节点标签是产生式左部的非终结符。
- 它的子节点从左到右对应产生式右部的符号。
- 根节点通常是开始符号。

例如 $E \Rightarrow E + T$ 在树中表现为：父节点 E ，三个子节点依次为 $E, +, T$ 。

6.2 叶子、产出与边缘 [p34,35]

Yield / Frontier

分析树从左到右读出所有叶子节点，得到的符号串称为该树的**产出** (yield) 或**边缘** (frontier)。它构成一个句型；如果全是终结符，就是一个句子。

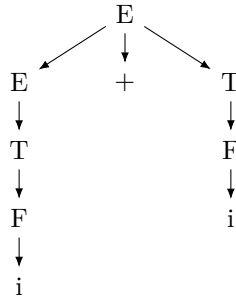
分析树与推导的关系

普通推导可能有许多替换顺序，但分析树不记录这些顺序。对一棵固定分析树而言，可以唯一恢复出一条最左推导，也可以唯一恢复出一条最右推导。因此最左/最右推导常用于判定分析树是否

唯一。

6.3 小型分析树示意

下面用一个简化树表示 $E \rightarrow E + T$ 、 $T \rightarrow F$ 、 $F \rightarrow i$ 等规则如何组合：



它的叶子从左到右是 $i + i$ ，因此 yield 是 $i + i$ 。

7 二义性与消歧 (第 36–41 页)

7.1 二义性的基本问题 [p36,37]

考虑文法：

$$G(E) : \quad E \rightarrow i \mid E + E \mid E * E \mid (E)$$

句子 $(i * i + i)$ 可以有两种结构：

$$(i * i) + i \quad \text{或} \quad i * (i + i)$$

这对应两棵不同的分析树，也意味着同一程序片段可能有两种语义。

为什么程序语言不喜欢二义性

如果一个程序字符串有两棵不同分析树，它就可能有两种不同含义。编译器必须给出确定行为，因此大多数 parser 更希望文法无二义，或者至少通过额外规则把多余分析树排除掉。

7.2 文法二义性与语言二义性 [p38]

文法二义性 (Ambiguity of Grammar)

如果某个文法存在一个句子，该句子对应多于一棵不同的分析树，则该文法是二义的。

语言二义性 (Ambiguity of Language)

如果一个语言不存在任何无二义文法能生成它，则这个语言是二义语言，也称**固有二义** (inherently ambiguous) 语言。

注意：某个文法 G 有二义性，不代表 $L(G)$ 这个语言本身固有二义。可能存在另一个文法 G' ，满足 $L(G) = L(G')$ ，但 G' 无二义。

7.3 二义性判定的困难 [p39]

课件指出：判断任意文法是否二义是**不可判定问题** (undecidable problem)。也就是说，不存在一个算法，能在有限步骤内对所有输入文法都准确判断“是否二义”。

因此工程上通常采用：

- 人工重写文法，尽量消除二义性。
- 使用二义文法，但配合**消歧规则** (disambiguating rules) 丢弃不想要的分析树，例如 YACC 中常见的优先级、结合性声明。
- 使用若干充分条件判断某类文法无二义；但充分条件不是必要条件，不能覆盖所有无二义文法。

7.4 用优先级和结合性消除表达式二义性 [p40,41]

表达式文法的二义性通常来自两个问题：

- **优先级** (precedence)：例如 $*$ 是否比 $+$ 先结合。
- **结合性** (associativity)：例如 $a - b - c$ 是 $(a - b) - c$ 还是 $a - (b - c)$ 。

一种无二义写法是：

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid i$$

解释：

- E 层处理 $+$ ， T 层处理 $*$ ，所以 $*$ 优先级高于 $+$ 。
- $E \rightarrow E + T$ 和 $T \rightarrow T * F$ 使用左递归，通常表达左结合。
- 括号在 $F \rightarrow (E)$ 中出现，能显式改变默认优先级。

8 乔姆斯基文法体系 (第 42–49 页)

8.1 四类文法总览 [p42,43]

Noam Chomsky 在 1956 年建立形式语言体系，将文法按产生式限制分为 0、1、2、3 四类。它们都可写作 $G = (V_T, V_N, S, \delta)$ ，差异在于产生式允许的形式不同。



包含关系可理解为：

$$\text{Regular} \subset \text{Context-Free} \subset \text{Context-Sensitive} \subset \text{Unrestricted}$$

8.2 Type 0: 无限制文法 [p44]

0 型文法 (Unrestricted Grammar / Phrase-Structure Grammar)

若每条产生式 $\alpha \rightarrow \beta$ 满足：

$$\alpha \in (V_N \cup V_T)^*, \quad \alpha \neq \varepsilon, \quad \beta \in (V_N \cup V_T)^*$$

则该文法是 0 型文法。

特点：

- 左部可以是复杂符号串，不要求单个非终结符。
- 右部可以变长、变短，甚至为空。
- 与图灵机 (Turing Machine) 等价，表达能力最强。
- 推导过程中字符串长度可以反复扩张和收缩，达到目标串前的推导步数没有固定上界。

课件例子：

$$aA \rightarrow aBCd, \quad aBcd \rightarrow aE, \quad A \rightarrow \varepsilon$$

8.3 Type 1: 上下文有关文法 [p45]

1 型文法 (Context-Sensitive Grammar, CSG)

若每条产生式 $\alpha \rightarrow \beta$ 满足：

$$\alpha \in (V_N \cup V_T)^*, \quad \alpha \neq \varepsilon, \quad \beta \in (V_N \cup V_T)^*, \quad |\alpha| \leq |\beta|$$

并且通常不允许空产生式 (除非是受限的开始符号特例)，则为上下文有关文法。

典型形式：

$$\alpha A \beta \rightarrow \alpha \gamma \beta$$

表示非终结符 A 只有在左边是 α 、右边是 β 这个上下文中，才能被替换为 γ 。

特点：

- 若不考虑特殊空串规则，推导串长度只能不减，因此推导步数受目标串长度约束。
- 与**线性有界自动机** (Linear Bounded Automaton, LBA) 相关。
- 表达能力比 CFG 强，但解析复杂且效率低。

8.4 Type 2: 上下文无关文法 [p46]

2 型文法 (Context-Free Grammar, CFG)

若每条产生式形如

$$A \rightarrow \alpha, \quad A \in V_N, \quad \alpha \in (V_N \cup V_T)^*$$

即左部是单个非终结符，则为上下文无关文法。

特点：

- 替换 A 时不看上下文，因此称“上下文无关”。
- 对应**非确定下推自动机** (Nondeterministic Pushdown Automaton, NDPDA)。
- 能描述递归和嵌套结构，是语法分析的核心工具。

例子：

$$S \rightarrow aSb \mid ab$$

生成

$$L = \{a^n b^n \mid n \geq 1\}$$

该语言不是正则语言，但可以由 CFG 描述。

8.5 Type 3: 正则文法 [p47,48]

3 型文法 (Regular Grammar)

若每条产生式形如

$$A \rightarrow aB \quad \text{或} \quad A \rightarrow a$$

其中 $A, B \in V_N$, $a \in V_T \cup \{\varepsilon\}$ ，则为正则文法。开始符号可在受限条件下推出 ε 。

特点：

- 左部是单个非终结符，右部最多是一个终结符加一个非终结符。
- 与**有限自动机** (Finite Automaton, FA) 等价。
- 推导串长度通常每步增加 1，因此表达能力最弱。

课件例子：

$$A \rightarrow 1A \mid 0$$

对应正则表达式：

$$1^*0$$

8.6 四类文法对比 [p49]

类型	名称	产生式限制	对应模型 / 用途
0 型	无限制文法	$\alpha \rightarrow \beta$, 左部非空, 几乎不限制长度变化	图灵机; 理论表达能力最强。
1 型	上下文有关文法	$ \alpha \leq \beta $, 基本不收缩	线性有界自动机; 表达上下文约束但解析效率差。
2 型	上下文无关文法	$A \rightarrow \alpha$, 左部单个非终结符	下推自动机; 编译器语法分析核心。
3 型	正则文法	$A \rightarrow aB$ 或 $A \rightarrow a$	有限自动机; 词法分析核心。

9 实践中的语法分析与 AST (第 50–55 页)

9.1 为什么编译器仍大量使用 CFG [p50]

课件指出，大多数现代程序设计语言整体上并不是纯上下文无关语言，甚至很多约束也不是上下文有关文法适合直接处理的。例如：

- 变量必须先声明再使用。
- 函数调用参数数量和类型必须匹配。
- 标识符作用域、类型检查、继承关系等需要语义信息。

但 CFG 仍被广泛用于编译器：

- CFG 非常适合描述表达式和语句的递归结构。
- CSG parser 理论和实现都复杂，效率不适合常规编译。
- CFG 的构造和解析技术成熟高效。
- 剩余的上下文相关约束通常放到语义分析 (Semantic Analysis) 阶段处理。

工程分工

在程序语言实现中，常见分工是：

正则语言 → 词法分析 上下文无关语言 → 语法分析 上下文/类型约束 → 语义分析

9.2 Parsing 到底做什么 [p51]

语法分析可以概括为：基于给定文法处理输入串，如果输入串属于该文法生成的语言，就构造它的推导表示；否则报告错误。

两个子任务：

1. 判断一个字符串是否能由文法推导出来。

2. 构造推导的表示，并传给编译器下一阶段。

推导表示可以是：

- 分析树 (Parse Tree)：完整保留文法推导细节。
- 抽象语法树 (Abstract Syntax Tree, AST)：省略不影响语义的细节，更适合后续编译阶段。

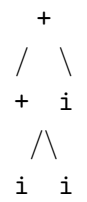
9.3 Parse Tree vs AST [p52]

抽象语法树 (Abstract Syntax Tree, AST)

AST 是 parse tree 的缩略表示。它在不改变程序含义的前提下，删除括号、某些中间非终结符、纯语法辅助节点等推导细节，只保留语义上重要的结构。

对比项	Parse Tree	AST
关注点	文法如何一步步推导出输入串	程序真正的语义结构
节点数量	多，包含所有终结符、非终结符和标点	少，省略无意义中间节点
括号与标点	通常保留	只在影响语义时保留
适用阶段	语法分析解释、教学、调试	语义分析、中间代码生成、优化

例如表达式 $(i + i) + i$ 的 parse tree 会包括括号、 E, T, F 等许多中间节点；AST 通常只保留两个 $+$ 节点和三个 i 叶子：



9.4 课程总结 [p53–55]

本讲的核心知识点可以压缩为：

- 语法分析输入 token 流，判断它是否属于文法生成的语言。
- 文法 $G = (V_T, V_N, S, \delta)$ 是定义形式语言的数学工具。
- CFG 的产生式形如 $A \rightarrow \alpha$ ，适合描述程序语言的递归语法结构。
- 推导从开始符号出发，连续应用产生式生成句型 and 句子。
- Parse tree 展示字符串如何由文法推导而来。
- 文法二义性意味着某个句子有多棵 parse tree，通常要用优先级、结合性或文法重写消除。
- AST 是程序语法结构的抽象表示，比 parse tree 更适合后续编译阶段。
- 延伸阅读可参考 Dragon Book 第 2.4、4.1.1、4.2、4.3 节。

10 练习题与参考答案

练习 1：区分词法分析和语法分析

源程序片段为：

$x * \%$

词法分析器可能把它识别为 $\text{id}(x)$, $\text{op}(*)$, $\text{op}(\%)$ 。请问为什么它仍然可能被 parser 判为非法？

答案：词法分析只判断每个字符片段能否构成合法 token；语法分析要判断 token 序列是否符合表达式或语句文法。 $\text{id} * \%$ 中 $*$ 后面缺少合法操作数， $\%$ 也不能直接作为操作数，因此 token 各自合法，但组合不合法。

练习 2：写出文法四元组

给定文法：

$$E \rightarrow E + T \mid T, \quad T \rightarrow T * F \mid F, \quad F \rightarrow (E) \mid \text{id}$$

写出它的 V_T, V_N, S 。

答案：

$$V_N = \{E, T, F\}, \quad V_T = \{+, *, (,), \text{id}\}, \quad S = E$$

δ 是题目给出的全部产生式集合。

练习 3：由文法推导字符串

文法 $G[A] : A \rightarrow c \mid Ab$ 。推导字符串 $cbbb$ 。

答案：

$$A \Rightarrow Ab \Rightarrow Abb \Rightarrow Abbb \Rightarrow cbbb$$

因此 $cbbb \in L(G)$ 。

练习 4：判断句型 and 句子

设 $S \Rightarrow^* aAb \Rightarrow aabb$ ，其中 A 是非终结符， a, b 是终结符。问 aAb 和 $aabb$ 分别是什么？

答案： aAb 是句型，因为它可由开始符号推导出，但仍含非终结符 A 。 $aabb$ 是句子，因为它只含终结符。

练习 5：最左推导

使用文法

$$E \rightarrow T \mid E + T, \quad T \rightarrow F \mid T * F, \quad F \rightarrow (E) \mid i$$

给出 $(i * i + i)$ 的最左推导。

答案：

$$\begin{aligned} E &\Rightarrow T \Rightarrow F \Rightarrow (E) \Rightarrow (E + T) \Rightarrow (T + T) \\ &\Rightarrow (T * F + T) \Rightarrow (F * F + T) \Rightarrow (i * F + T) \\ &\Rightarrow (i * i + T) \Rightarrow (i * i + F) \Rightarrow (i * i + i) \end{aligned}$$

练习 6：解释二义性

为什么文法

$$E \rightarrow i \mid E + E \mid E * E \mid (E)$$

对表达式 $(i * i + i)$ 是二义的？

答案：因为该文法没有规定 $*$ 和 $+$ 的优先级，也没有规定结合性。 $(i * i + i)$ 可以被解释为 $(i * i) + i$ ，也可以被解释为 $i * (i + i)$ ，对应两棵不同 parse tree。因此文法对该句子二义。

练习 7：改写无二义表达式文法

写出一个能体现 $*$ 优先级高于 $+$ ，并且二者左结合的表达式文法。

答案：

$$E \rightarrow E + T \mid T, \quad T \rightarrow T * F \mid F, \quad F \rightarrow (E) \mid i$$

E 层处理加法， T 层处理乘法，因此乘法优先级更高；左递归结构体现左结合。

练习 8：判断文法类型

判断以下文法属于哪一类乔姆斯基文法：

$$S \rightarrow aSb \mid ab$$

答案：它是 2 型文法，即上下文无关文法。每条产生式左部都是单个非终结符 S ，右部是终结符和非终结符组成的串。它生成 $L = \{a^n b^n \mid n \geq 1\}$ ，该语言不是正则语言。

练习 9：为什么不用 CSG 做主要语法分析

既然上下文有关文法比 CFG 更强，为什么编译器主要使用 CFG？

答案：CSG 表达能力更强，但解析复杂、效率低，不适合作为常规编译器主力工具。CFG 已经足够描述表达式、语句、块结构等主要递归语法，而且解析技术成熟高效。变量声明、类型匹配、作用域等上下文相关约束通常交给语义分析阶段处理。

练习 10：Parse Tree 与 AST

为什么编译器后续阶段通常更喜欢 AST，而不是完整 parse tree？

答案：Parse tree 完整展示文法推导，包含许多中间非终结符、括号、标点等语法细节。AST 删除不影响语义的细节，只保留程序结构和语义关系，因此更紧凑、更适合语义分析、中间代码生成和优化。

考前速记

- Parser 输入 token 流，输出 parse tree 或 AST。
- Grammar: $G = (V_T, V_N, S, \delta)$ 。
- CFG: $A \rightarrow \alpha$ ，左部是单个非终结符。
- $S \Rightarrow^* \alpha$: α 是句型；若 $\alpha \in V_T^*$ ，则是句子。

- Parse tree 的叶子从左到右组成 yield/frontier。
 - 二义文法：某个句子有多棵 parse tree。
 - 表达式消歧：靠优先级、结合性，或重写为 $E/T/F$ 分层文法。
 - Chomsky：正则 $\subset$ 上下文无关 $\subset$ 上下文有关 $\subset$ 无限制。
-